Calculating classification loss and accuracy

2) Implement evaluation utilities

Loss function: Cross entropy loss

Input text message

"You won the lottery" -> LLM -> [3.5983, -3.9902] -> [0.99, 0.01] -> 0 (not spam)

"Do you have time" -> LLM -> [-3.9846, 5.2945] -> [0.01, 0.99] -> 1 (spam)

outputs corresponding to last row (token)

1) convert outputs to softmax probabilities

predicted labels

Index position: 0 1

(not spam) (spam)

2) Locate index position of highest probability value in each row/vector, which is done via argmax function

Dataset (True) Predicted

-> loss function (L)

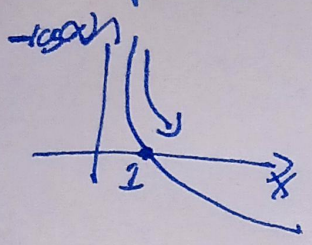
-> ∂L/∂w -> Update weights

(Unchanged — ∂L/∂w)

y_i, p_i

$-\sum y_i \log p_i$

true — predicted



Loss Penalties; Cross Entropy Loss

One: Non spam (0,0)

Predicted; [0.8, 0.2]

Non_spam ↳ spam

Cross entropy loss $= -\sum_{i=1}^{C} y_i \log(p_i)$

$= -[1 * \log(0.8) + 0 * \log(0.2)]$

$= 0.0231$

What if predicted was [1, 0]

$= -[1 * \log(1) + 0 * \log(0)]$

$= 0$ ✓

Step 6: Finetune model on supervised data

1) For each training epoch
2) For each batch in training set
3) Reset loss gradients from previous epoch
4) Calculate loss on current batch
5) Backward pass to calculate loss gradients
6) Update model weights using loss gradients
7) Print training and validation set losses
8) Now, calculate classification accuracy

↳ uses steps for training deep NNs in PyTorch

$w_{new} = w_{old} - \epsilon \frac{\partial L}{\partial w}$